DineLine AI

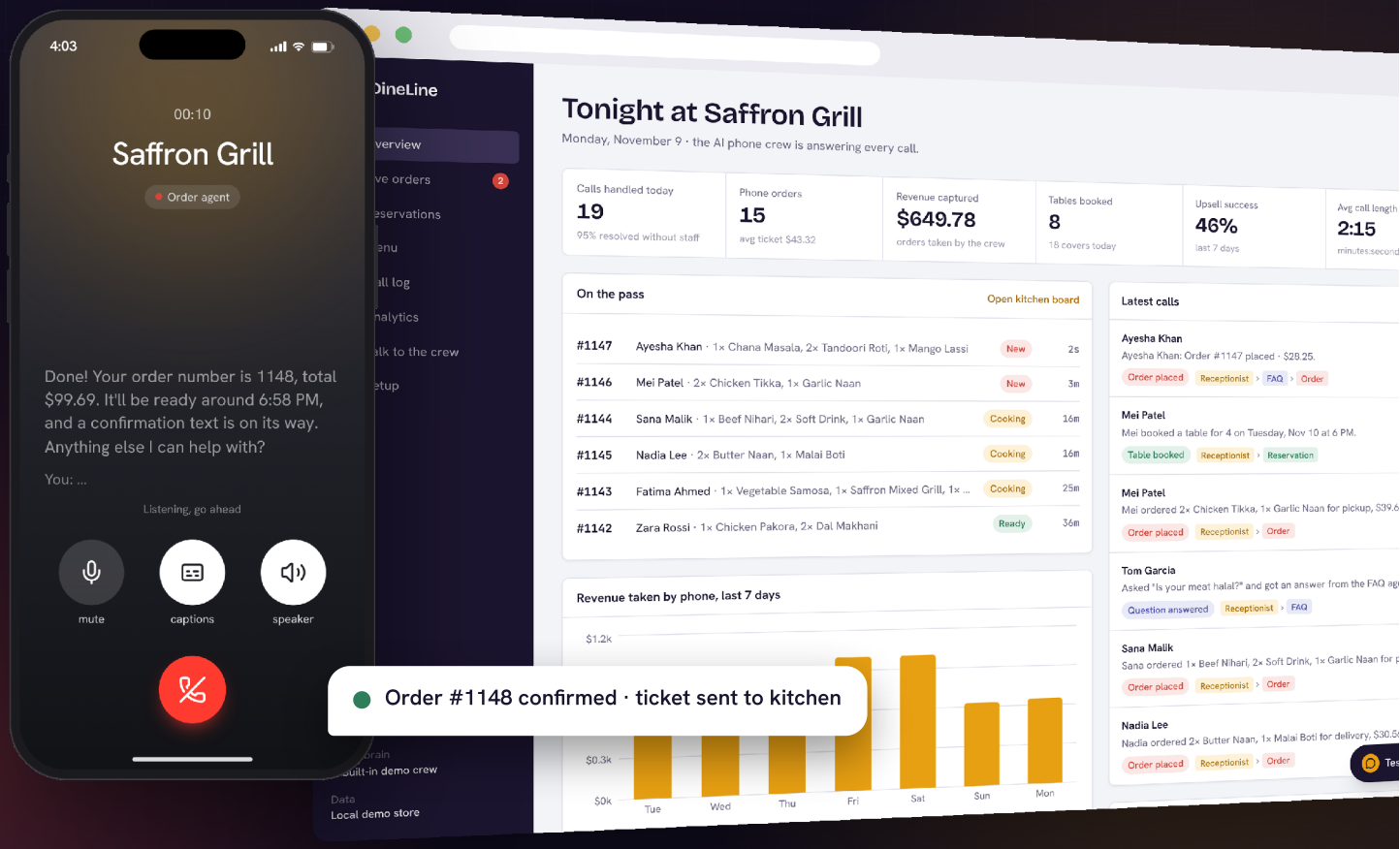
TECHNICAL DELIVERY REPORT

MULTI-AGENT VOICE AI · CUSTOM CRM

The AI phone crew that runs a restaurant's calls

Four specialised voice agents answer every call, take pickup and delivery orders, book tables and answer menu questions, while a fully automated CRM runs the kitchen, the floor and the owner's reports.

- Receptionist
- Order agent
- Reservation agent
- FAQ agent



4	15	7	9
AI agents with handoffs	business tools	n8n workflows	automated CRM screens

Muhammad Hassan
AI Automation Engineer & Full Stack Developer

Demo client
Saffron Grill, New York

Project details

ITEM	DETAIL
Project	DineLine AI: a multi-agent AI phone crew and fully automated custom CRM for restaurants
Demo client	Saffron Grill, New York (full-service restaurant with pickup, delivery and table bookings)
Technical lead	Muhammad Hassan, AI Automation Engineer & Full Stack Developer
Voice and AI	Vapi Squads (four assistants), Deepgram Nova-3 speech-to-text, OpenAI GPT-4o-mini, natural Vapi voices
Core platforms	Next.js 16 custom CRM, n8n workflow automation, Supabase Postgres with Realtime
Integrations	Twilio SMS and WhatsApp, Vapi tool calls and end-of-call webhooks, n8n webhooks, REST APIs
Scope of delivery	4 voice agents, 15 business tools, 7 n8n workflows, a 9-screen CRM, a 9-table database, setup scripts, documentation and tests
Status	Delivered All scope items complete

Contents

1 At a glance
2 The problem it solves
3 Original plan vs delivered
4 System architecture
5 Technology stack
6 The multi-agent voice crew
7 Business tools
8 A call from start to finish
9 The custom restaurant CRM
10 n8n automation workflows
11 Data model
12 Cost efficiency
13 Speed and reliability
14 Safety, security and data
15 Testing and verification
16 Repository and setup
17 Delivery and handover
18 Key capabilities and impact

1 At a glance

> **In plain words:** this is a restaurant phone line that never misses a call. Four AI agents answer, take the order or booking the way a trained host would, and put it straight onto the kitchen screen or the floor plan. The customer gets a text, the owner gets a nightly report, and the staff only cook and serve.

DineLine AI replaces the restaurant's phone line with a crew of four specialised AI agents. A Receptionist greets every caller and hands them to the right specialist for orders, reservations or questions. Every order lands on a live kitchen board, every booking on the floor plan, and every customer gets a text confirmation, with no staff involvement.

4 specialised voice agents in one Vapi Squad	**15** business tools shared by voice, chat and n8n	**7** n8n workflows: tools, texts, reminders, reports	**9** CRM screens, incl. phone simulator
76–86% fewer input tokens per AI call vs an all-in-one agent	**≤ 9 s** hard cap on any reply; fast mode answers in 0.07 s	**~$0.09** estimated cost per call minute (Vapi stack)	**0** staff actions needed from call to kitchen ticket

Token figures are measured from the real prompts and tool schemas. The per-minute cost is an estimate from Vapi's published rates; see section 12.

What the caller experiences

- Answered on the first ring, 24/7, greeted by name if they've ordered before.
- Orders taken from the live menu, with sizes, special requests, sold-out swaps and one natural upsell.
- Delivery zone checked, total read back with tax and fees, order number and ready time given.
- Tables booked, moved or cancelled; nearest times offered when a slot is full; waitlist when it's fully booked.
- Menu, allergen, halal, parking and hours questions answered from real data, never guessed.
- An SMS or WhatsApp confirmation seconds after the call, and a reminder before their booking.

2 The problem it solves

> **In plain words:** restaurants lose money every time the phone rings during a rush. DineLine answers every call, so no order or booking slips away and the team stays with the guests in front of them.

Independent restaurants lose revenue on the phone every service. The phone rings hardest exactly when the team is busiest, and every missed or rushed call is a lost order or an empty table.

Missed calls at peak hours

During the dinner rush nobody can stop to answer. Callers hang up and order from a competitor or a delivery app that takes a 15–30% commission.

Slow, error-prone phone orders

Orders are scribbled on paper, sizes and allergies get missed, totals are calculated by hand, and the kitchen gets a ticket it has to decipher.

The same questions, all day

"Are you open?", "Do you deliver to me?", "Is it halal?", "Is it nut-free?" Staff answer them dozens of times a day, sometimes wrongly on allergens.

No record, no follow-up

Calls leave no trace: no transcript, no customer history, no confirmation text, no reminder before a booking, no data on what callers want.

What DineLine changes

BEFORE	WITH DINELINE AI
Calls missed during service	Every call answered instantly, unlimited calls in parallel
Handwritten order tickets	Structured ticket on the kitchen board seconds after the call, with sizes, notes and allergies
Totals worked out by hand	Priced by the system: line items, tax, delivery fee and minimum checked automatically
Upsell depends on who answers	One natural, context-aware upsell on every order (a drink, bread with curries, or dessert)
Bookings in a paper diary	Live floor plan with automatic table assignment, waitlist and reminders
No customer history	Returning callers recognised by number, greeted by name, offered "same as last time"
No insight into calls	Full transcripts, summaries, outcomes and analytics for every conversation

3 Original plan vs delivered

Delivery status: complete. The project brief set the scope before any code was written. All 11 scope items were delivered: 6 as planned, 3 extended beyond the brief and 2 added on top of it.

AREA	PLANNED	DELIVERED	STATUS
Voice agents	4 agents in a Vapi Squad	Receptionist, Order, Reservation and FAQ agents with handoffs in every direction; one setup command creates tools, assistants and the squad	Delivered
Agent tools	Tool calls for menu, orders and bookings	15 tools, including price_order (read-back before placing), find/modify/cancel booking, waitlist and order status	Extended
n8n workflows	6 core + 1 optional	All 7, importable, each verified running in a real n8n instance	Delivered
Database	7 Supabase tables	9 tables incl. order line items and a single settings record, with Realtime, triggers, sequence and row-level security	Extended
Kitchen dashboard	5 screens	A full custom CRM with 9 screens: overview, live orders, reservations, menu, call log, analytics, crew console, setup and a phone simulator	Extended
Notifications	SMS or WhatsApp confirmation	Order, status, booking, cancellation and reminder texts via n8n + Twilio or directly from the app, with a visible outbox	Delivered
Post-call	Transcript, summary, outcome	End-of-call reports stored with agents involved, outcome, cost, recording link and linked order or booking	Delivered
Daily report	Optional	Nightly WhatsApp summary for the owner (workflow 07)	Delivered
Testing in the browser	Not in the plan	Crew console with live trace, hands-free voice calls and a realistic phone screen with live captions	Added
Resilience	Not in the plan	Built-in rule-based crew, 9-second reply cap, retries and a circuit breaker for slow AI providers	Added
Voice platform	Vapi; LiveKit optional	Provider-agnostic tool layer shared by Vapi, n8n and the web console; any voice platform plugs in without code changes	Delivered

4 System architecture: product overview

> **In plain words:** one picture of the whole product: how a call travels from the first ring to the kitchen, who does what along the way, and what the restaurant gets out of it.

DineLine AI

Autonomous restaurant phone crew · orders · reservations · menu answers · automated CRM

WORKFLOW ARCHITECTURE

1. **Call comes in** — Vapi number answers on the first ring, 24/7
2. **Receptionist** — Greets, recognises returning callers, finds intent
3. **Specialist agent** — Order, reservation or FAQ agent takes over
4. **Tools act** — Live menu, prices, delivery zone, free tables
5. **CRM updates** — Kitchen ticket or floor plan, caller history
6. **Customer texted** — SMS or WhatsApp via n8n and Twilio
7. **Owner report** — Nightly WhatsApp summary at 11 PM

THE SYSTEM

VOICE AGENT CREW

- **Receptionist (Aria)** — Answers, looks up the caller, routes in one turn
- **Order agent** — Sizes, add-ons, upsell, delivery check, exact read-back
- **Reservation agent** — Books, moves and cancels; alternatives and waitlist
- **FAQ agent** — Menu, allergens, hours, parking, catering

CUSTOM CRM · 9 SCREENS

Overview	tonight at a glance
Live orders	kitchen board
Reservations	floor plan, waitlist
Menu	prices, sold out
Call log	transcripts
Analytics	trends, top dishes
Crew console	live agent trace
Setup	integrations
Phone	caller simulator

INTEGRATIONS & AUTOMATION

Vapi Squads	voice, handoffs
Deepgram Nova-3	speech to text
GPT-4o-mini	reasoning
n8n	7 workflows
Twilio	SMS, WhatsApp
Supabase	9 tables, Realtime

SHARED BUSINESS LAYER

15 tools · 19 API routes · one rulebook for every channel

WHAT THE RESTAURANT GETS

- **24/7** — every call answered, even at peak rush
- **Exact** — totals with tax and fees, computed by code
- **0%** — commission on phone orders
- **Auto** — table assignment, no double bookings
- **Live** — kitchen board, floor plan and call log
- **~$0.25** — estimated AI cost per order call

Ring → receptionist → specialist → tools → CRM → text → report

Demo client: Saffron Grill, New York

4 System architecture: technical view

> **In plain words:** Vapi is the voice on the phone, the DineLine app is the brain that knows the menu, prices and tables, n8n is the messenger that sends texts and reports, and Supabase is the filing cabinet that keeps every record.

Voice handling and business logic are separated on purpose. Vapi owns the real-time conversation (speech, reasoning, voice and handoffs); the DineLine app owns the business rules and data; n8n owns everything that happens around the call.

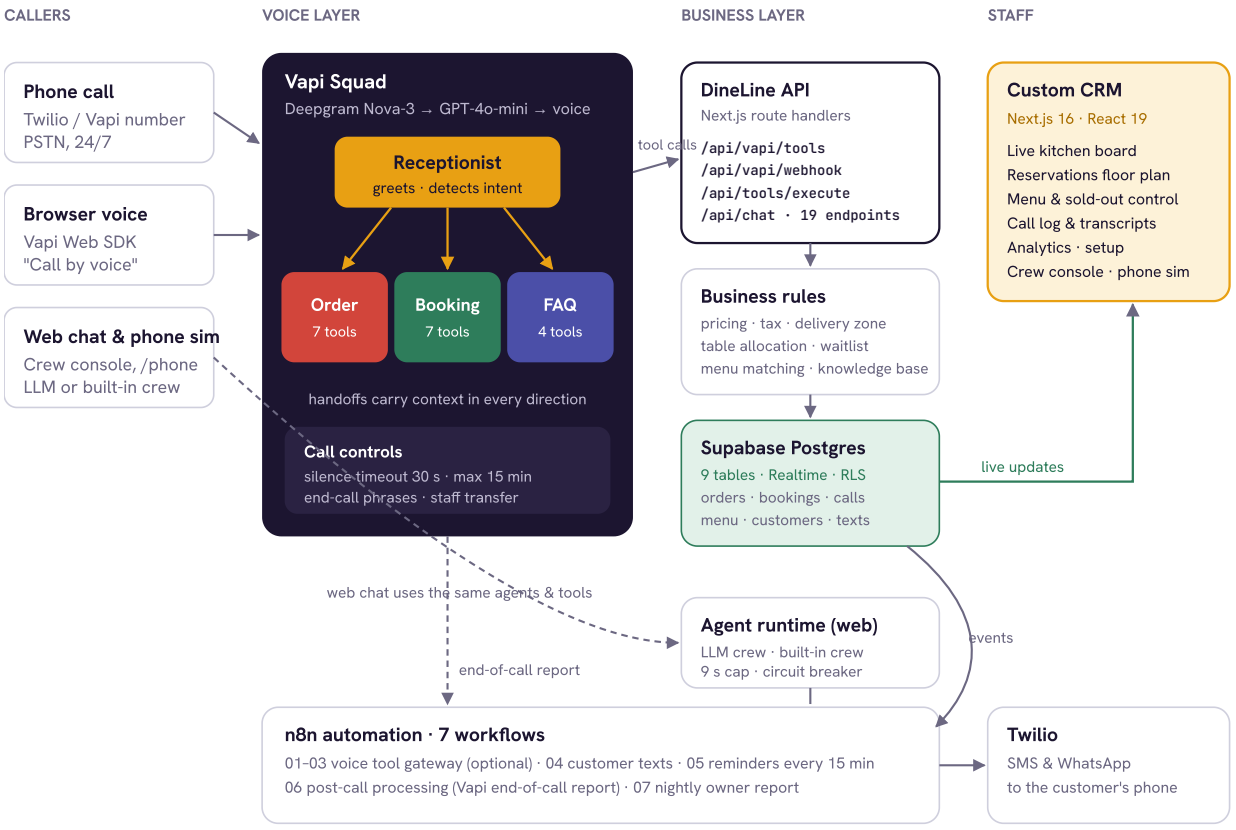


Figure 1. Production architecture. Solid arrows are live request paths; dashed arrows are asynchronous events.

Design decisions

- **One business layer for every channel.** Phone calls, browser voice, web chat and n8n all execute the same 15 tools. An order placed by phone, chat or workflow follows identical rules and lands on the same kitchen board.
- **Specialist agents instead of one giant prompt.** Each agent only carries its own instructions and tools, which keeps answers focused, reduces mistakes and cuts token cost per call (section 12).
- **Data comes from tools, not prompts.** The menu, prices, availability and policies are fetched live, so a dish marked sold out in the CRM is unavailable on the very next call, with no redeploy.
- **n8n where a client wants visibility.** Texts, reminders, logging and reports run as visual workflows the restaurant can see and change; the latency-critical tool path stays direct.
- **Provider-agnostic.** The agent runtime speaks any OpenAI-compatible API (OpenAI, Gemini, Groq, OpenRouter), and the data layer runs on Supabase or a local store behind one interface.

5 Technology stack

> **In plain words:** every service in the stack does one job well and has a free or low-cost tier, so the restaurant pays mainly for the minutes its phone line is actually in use.

Chosen for real-time voice quality, low per-call cost and a stack that clients can run, extend and hand over.

LAYER	TECHNOLOGY	ROLE IN THE SYSTEM
Voice orchestration	Vapi (Squads)	Real-time calls, turn-taking, interruptions, agent handoffs, tool calling, end-of-call analysis and recordings
Speech to text	Deepgram Nova-3	Low-latency transcription tuned for phone audio
Reasoning	OpenAI GPT-4o-mini	Fast, low-cost model for conversation and tool use on calls (configurable per assistant)
Voice	Vapi voices (configurable)	Natural speech output; ElevenLabs or other providers can be swapped in per assistant
Telephony	Twilio (or Vapi number)	Restaurant phone number, inbound calls, SMS and WhatsApp
Automation	n8n (self-hosted or cloud)	7 workflows: tool gateway, notifications, reminders, post-call processing, owner report
Database	Supabase (Postgres)	Orders, bookings, menu, customers, calls and texts; Realtime for the kitchen board; row-level security
CRM front end	Next.js 16, React 19, Tailwind CSS 4	Custom restaurant CRM with 9 screens, live updates and charts (Recharts)
API	Next.js route handlers (TypeScript)	19 endpoints for Vapi, n8n and the CRM; shared-secret protected webhooks
Web agent runtime	OpenAI-compatible LLM client	Runs the same four agents in the browser console (Gemini, OpenAI, Groq, OpenRouter) with a built-in rule-based fallback crew
Browser voice	Web Speech API, Vapi Web SDK	Hands-free voice calls and the phone simulator with live captions
Hosting	Vercel (app), n8n cloud or VPS	Serverless deploy; Supabase hosted

Why GPT-4o-mini on calls: restaurant calls are short, structured and tool-driven. A small, fast model gives sub-second reasoning and costs a fraction of larger models, while the tools enforce correctness (prices, availability, totals are computed by code, never by the model).

6 The multi-agent voice crew

> **In plain words:** like a good front-of-house team, each agent has one job. The receptionist answers and passes the caller to the order taker, the reservations host or the menu expert, and the caller never notices the handover.

Four assistants work as one team in a Vapi Squad. The caller hears one seamless conversation; behind it, each turn is handled by the specialist best suited to it.

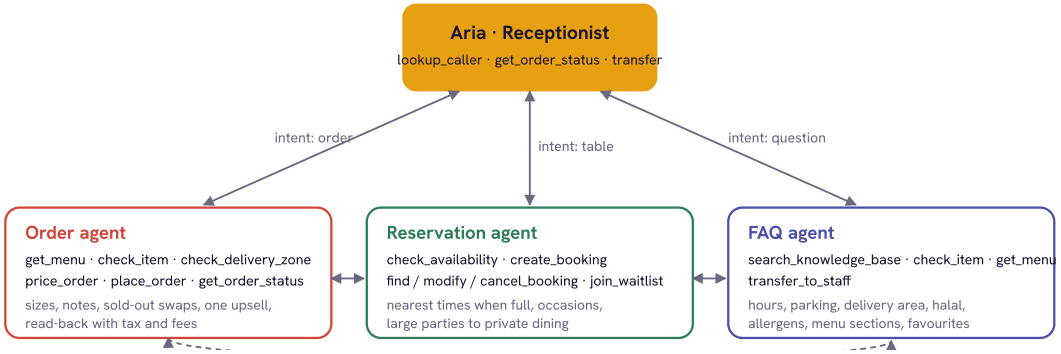


Figure 2. Agent handoff map. Every specialist can hand the caller to any teammate, so one call can cover a question, an order and a booking.

How the agents are instructed

- **Phone-first style.** One or two short sentences per reply, one question at a time, no lists or symbols, names and numbers confirmed back.
- **Never invent facts.** Menu items, prices, times and policies come only from tools; when a tool says no, the agent says so and offers the closest option.
- **Seamless handoffs.** Specialists pick up mid-conversation with the context gathered so far, never re-introducing themselves.
- **Menu like a host.** Never reads the whole menu: sections and three favourites first, then up to five dishes per section.
- **Escalation rules.** Complaints, refunds, catering quotes and parties above the largest table go to a human via live transfer.
- **Returning customers.** Greeted by name from caller ID; "same as last time" rebuilds their previous order.

Order agent conversation logic

1. Collect items; ask for size where a dish has sizes (biryani Regular or Family, karahi Half or Full); record notes such as "no onions" or spice level.
2. Swap sold-out items for available alternatives from the same section; tolerate typos and speech-recognition spellings ("lamp biryani", "garlic nan").
3. Pickup or delivery; for delivery, validate the ZIP against the delivery zone and the order against the minimum.
4. Price the draft and make exactly one context-aware upsell: a drink if there is none, bread with curries, otherwise dessert.
5. Read the order back with the exact total including tax and delivery fee; place it only after a clear yes, then give the order number and ready time.

7 Business tools

> **In plain words:** the agents talk; the tools do the work. Whenever a price, a table or a delivery area matters, a tool checks the real data, so the agent never guesses.

Fifteen tools turn the agents' decisions into real actions. They are defined once and served to Vapi, to n8n and to the web console, with OpenAI-compatible JSON schemas.

TOOL	USED BY	WHAT IT DOES
`lookup_caller`	Receptionist	Finds a returning customer by phone: name, order count, last order for "same as last time"
`get_menu`	Order, FAQ	Live menu by section with prices, sizes, availability and popular dishes
`check_item`	Order, FAQ	One dish: price, sizes, add-ons, allergens, dietary tags, spice, availability; alternatives when sold out
`check_delivery_zone`	Order	Validates the ZIP code; returns fee, minimum and delivery time, or offers pickup
`price_order`	Order	Prices a draft without saving: line totals, subtotal, tax, delivery fee, minimum check, upsell suggestions
`place_order`	Order	Creates the order, puts the ticket on the kitchen board, estimates the ready time, texts the confirmation
`get_order_status`	Receptionist, Order	"Where's my order?" by number or caller ID
`check_availability`	Reservation	Natural dates ("Friday", "tomorrow"), opening hours, last seating, table fit; nearest alternatives when full
`create_booking`	Reservation	Books the best-fitting table, honouring seating preference; returns a confirmation code and texts it
`find_booking`	Reservation	Finds upcoming bookings by code or caller ID
`modify_booking`	Reservation	Moves date, time or party size after re-checking availability
`cancel_booking`	Reservation	Cancels and texts the guest; flags waitlisted guests who could take the slot
`join_waitlist`	Reservation	Adds the caller to the waitlist with their position
`search_knowledge_base`	FAQ	Hours, location, parking, delivery area, payment, halal, allergens, dietary lists, catering, private dining
`transfer_to_staff`	Receptionist, FAQ	Live transfer to a staff number (Vapi transferCall) for complaints, refunds and large groups

Correctness by design: the model decides *what* to do; code decides *the numbers*. Prices, tax (8.875%), delivery fees, minimums, table capacity and opening hours are all computed by the tools, so the agent can never quote a wrong total.

8 A call from start to finish

> **In plain words:** from "hello" to a ticket on the kitchen screen in about two minutes, with nobody at the restaurant touching anything.

A real delivery order traced through the system. The caller speaks for about two minutes; everything after "yes" happens automatically.

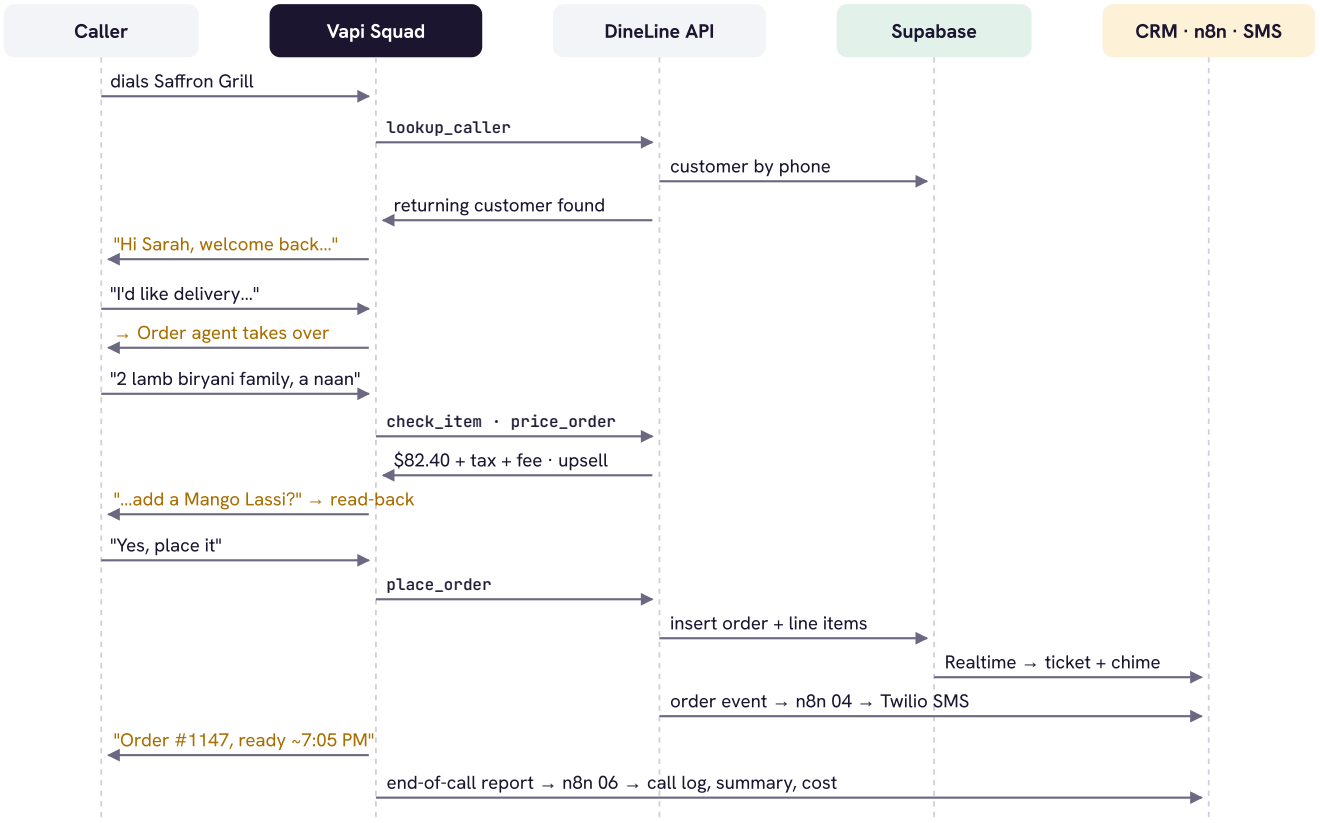


Figure 3. Sequence of a delivery order. Brown text is what the caller hears.

Kitchen

New ticket appears on the board with a chime: items, sizes, notes, address and total. One tap moves it through cooking, ready and out for delivery.

Customer

Gets the confirmation text with items, total and ready time, then automatic texts when the order is ready or out for delivery.

Owner

Sees the call in the call log with transcript, AI summary, agents involved, outcome and cost, and in tonight's analytics and owner report.

9 The custom restaurant CRM

> **In plain words:** the CRM is where the restaurant team works. The AI fills it in after every call; the kitchen, the host and the owner simply watch it and tap a button when something is ready.

Alongside the voice crew, a complete custom CRM was built for the restaurant: one place for the kitchen, front of house and owner. It is fully automated: the AI fills it, staff only act on it, and every staff action triggers the right customer message automatically.

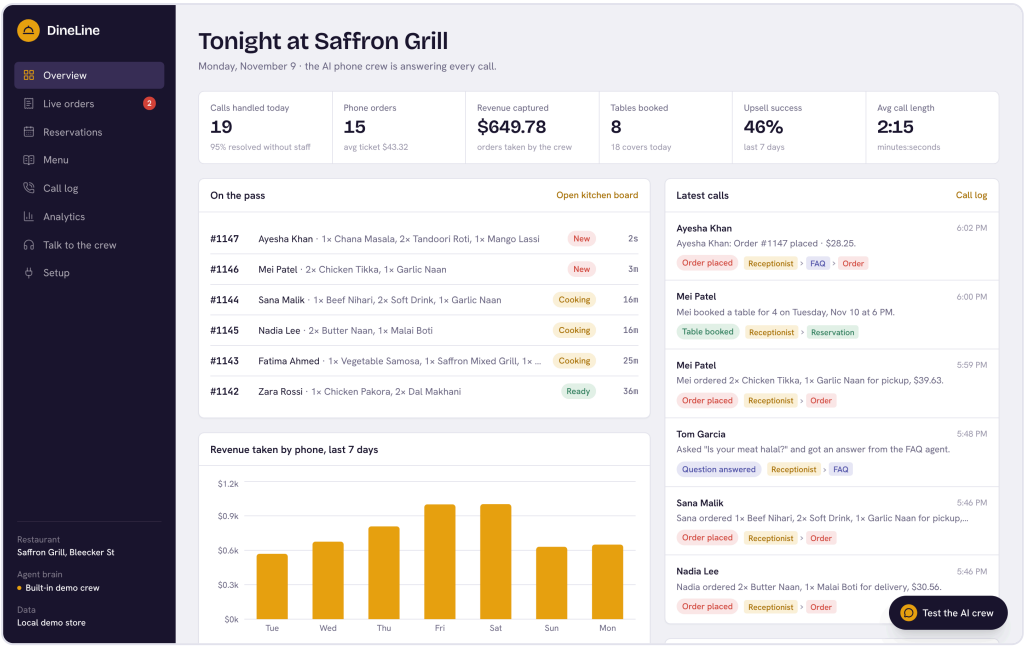


Figure 4. Overview: today's calls, phone orders, revenue captured, bookings, upsell rate and call length; the pass, latest calls and tonight's book.

Automations built into the CRM

WHEN THIS HAPPENS	THE CRM DOES THIS AUTOMATICALLY
The AI confirms an order	Ticket lands on the kitchen board with a chime; customer record and order count updated; confirmation text sent
Kitchen taps "Mark ready" / "Send out"	Customer gets a "ready for pickup" or "on its way" text
A dish is marked sold out	Agents stop offering it on the next call and suggest alternatives from the same section
A booking is made, moved or cancelled	Table assigned to the best fit, floor plan updated, guest texted the confirmation or cancellation
Host taps "Offer table" on a waitlisted guest	A free table is found and assigned automatically and the guest is texted
Two hours before a booking	Reminder text sent (n8n workflow 05), booking marked as reminded
A call ends	Transcript, AI summary, outcome, agents, duration and cost stored and linked to the order or booking
11 PM every night	Owner receives a WhatsApp report: calls, orders, revenue, covers, upsell rate, top dishes

Live orders: the kitchen board

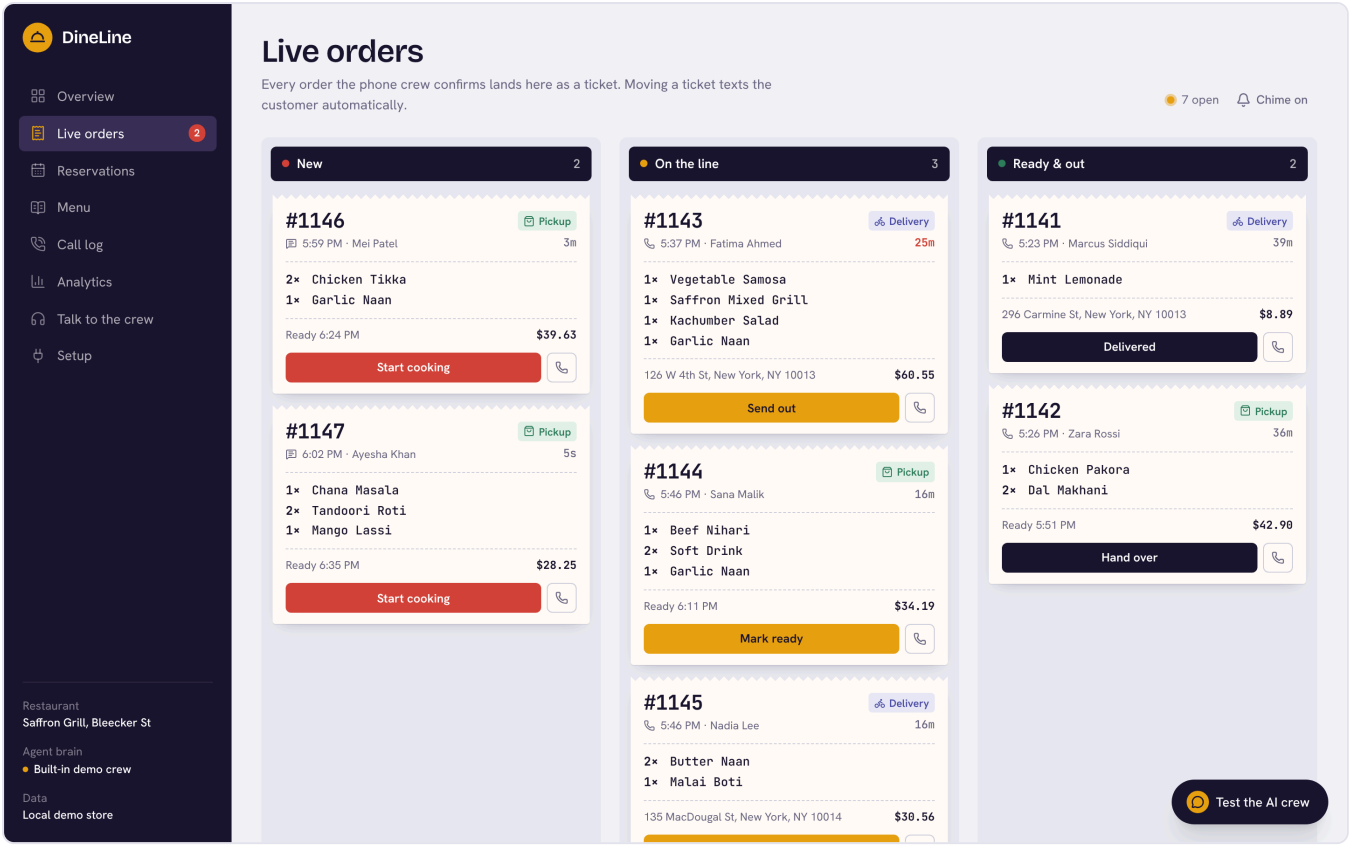


Figure 5. Orders arrive as thermal-style kitchen tickets in New → On the line → Ready & out, with age timers that turn red when a ticket is late.

Reservations: floor plan by time

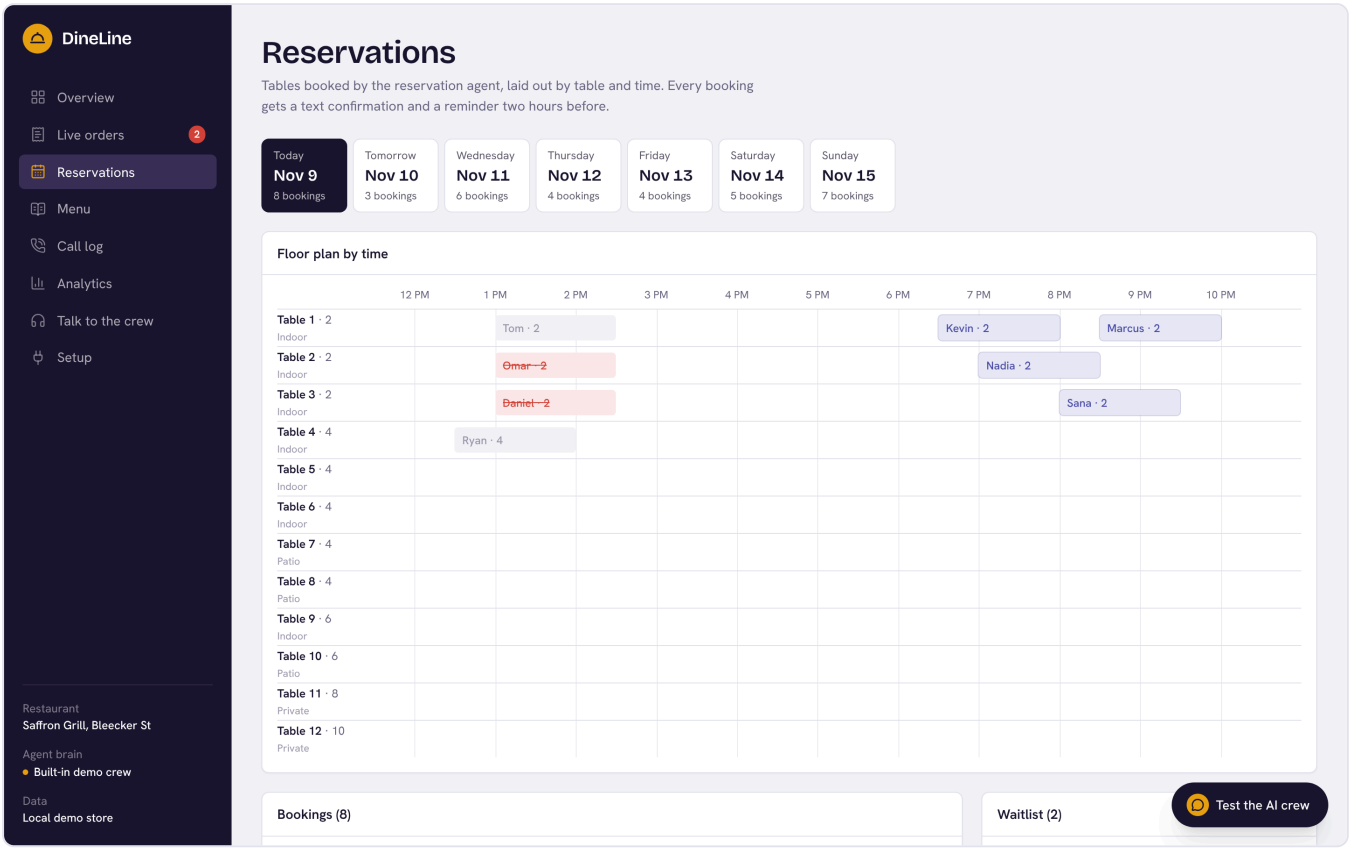


Figure 6. Seven-day reservation book with a table-by-time timeline, seat / no-show / cancel actions and the waitlist.

Call log and analytics

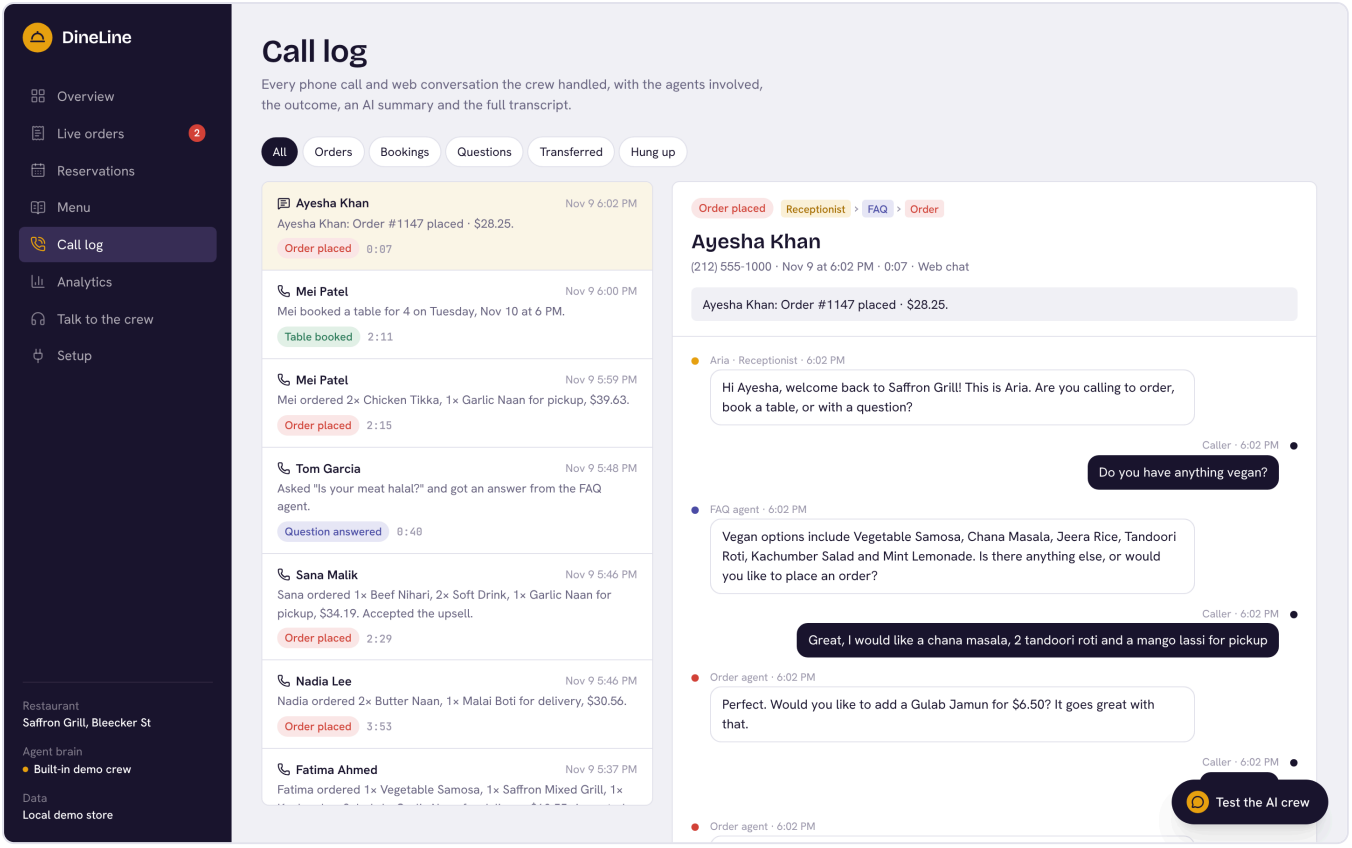


Figure 7. Every call and chat with outcome, agents involved, summary and the full transcript, coloured by agent.

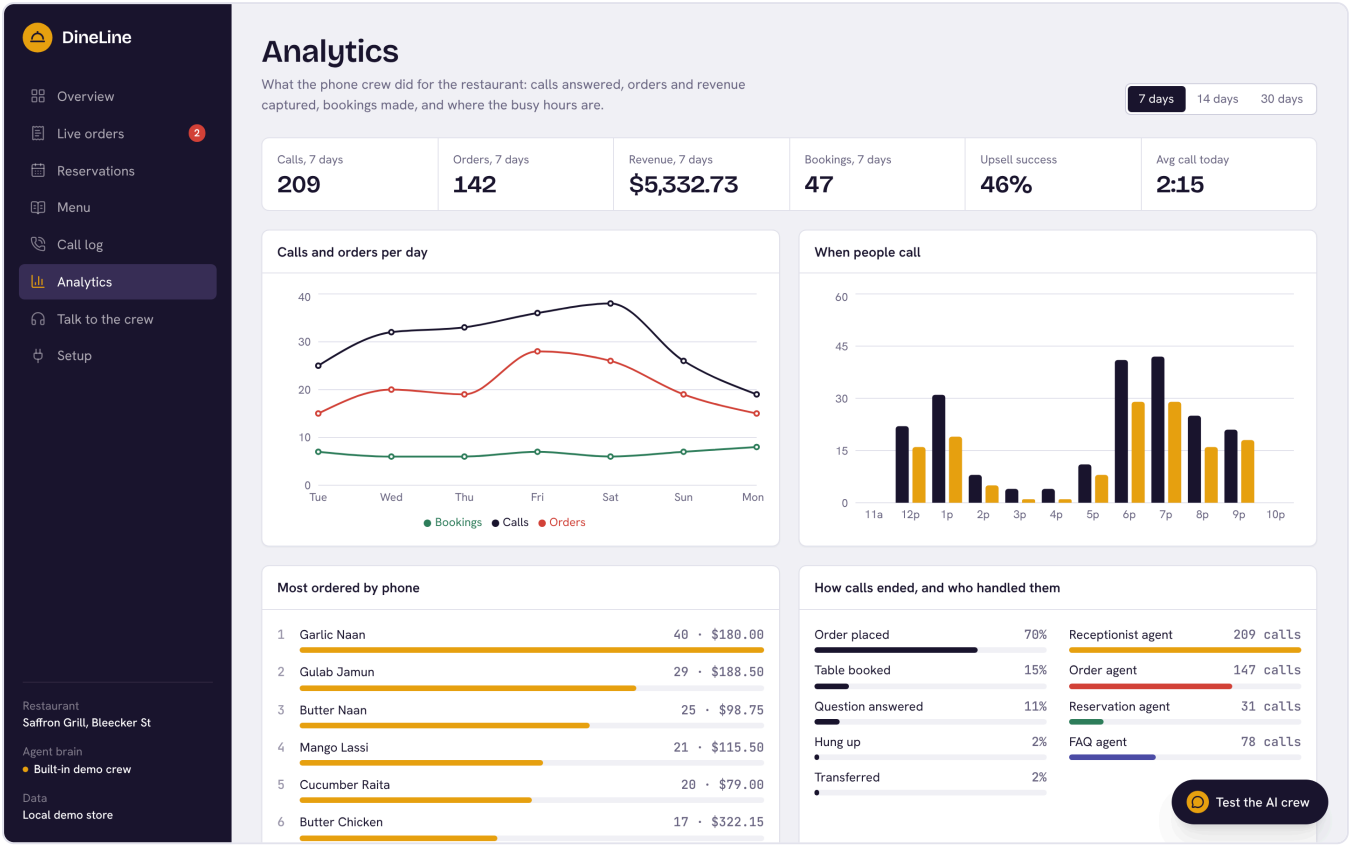


Figure 8. Calls, orders and bookings per day, busiest hours, most-ordered dishes, call outcomes and agent usage over 7, 14 or 30 days.

Menu control and the crew console

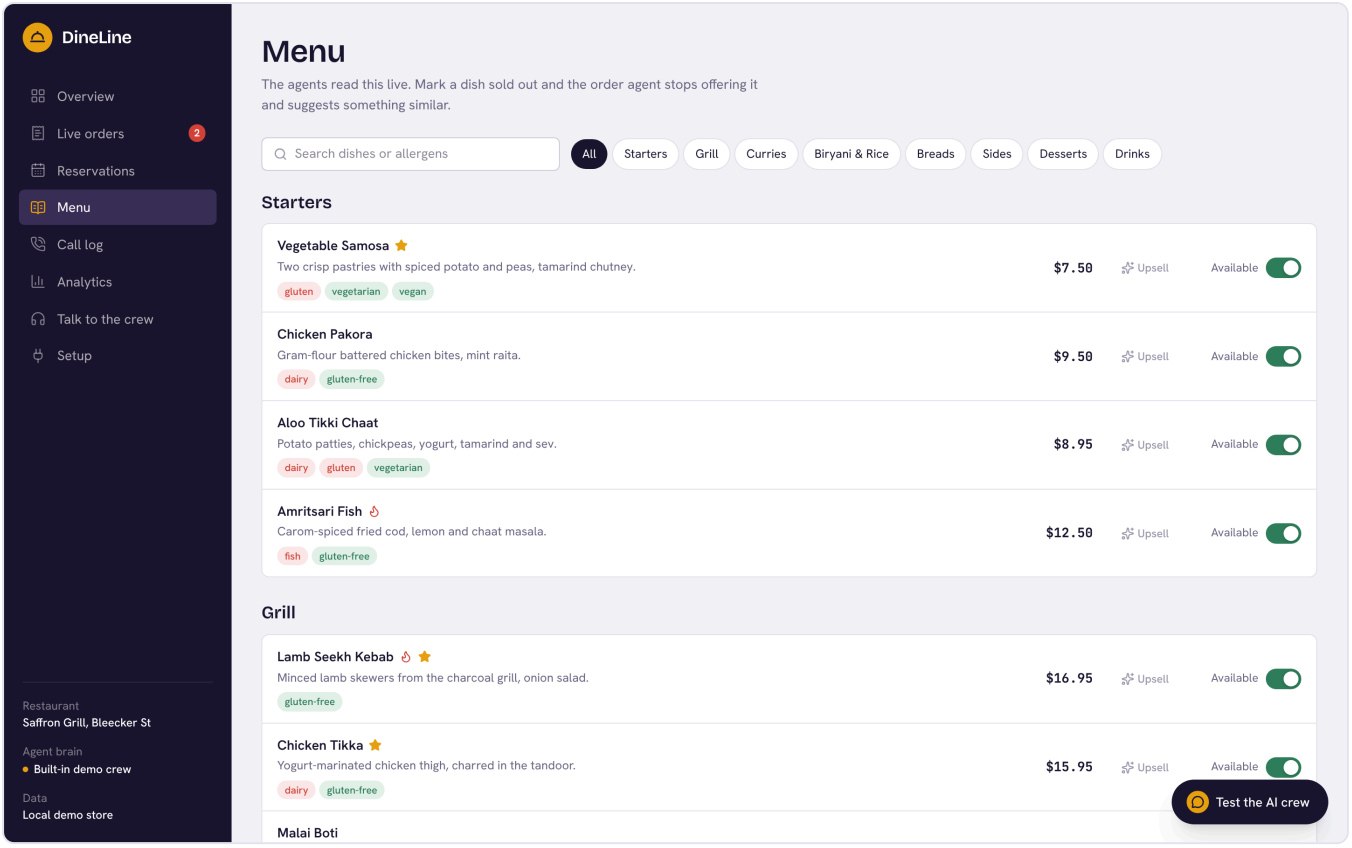


Figure 9. Menu manager: inline price edits, sold-out switch, upsell flags, allergens and dietary tags. Changes reach the agents on the next call.

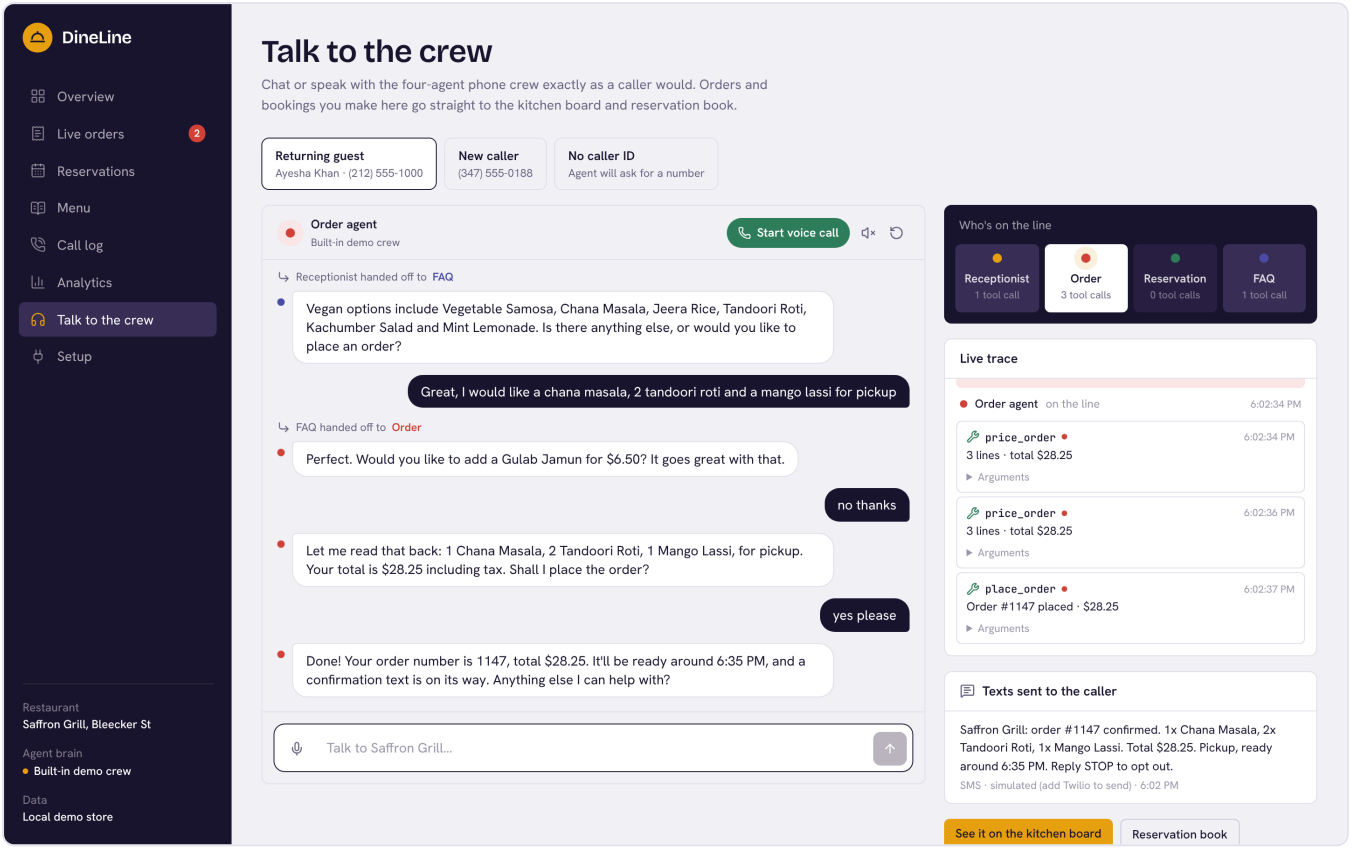
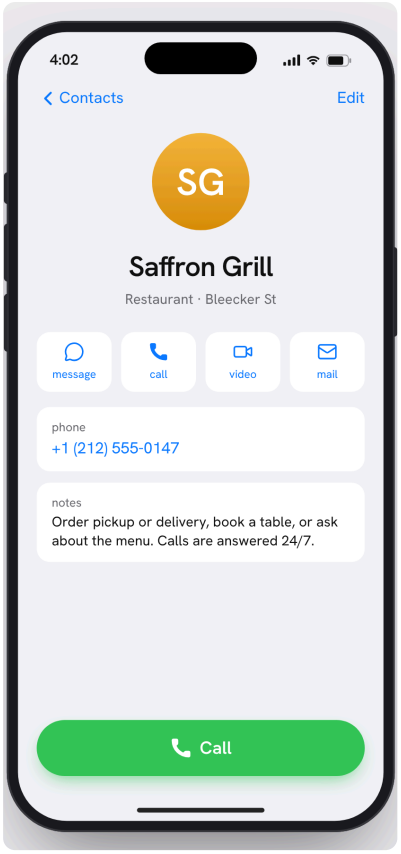


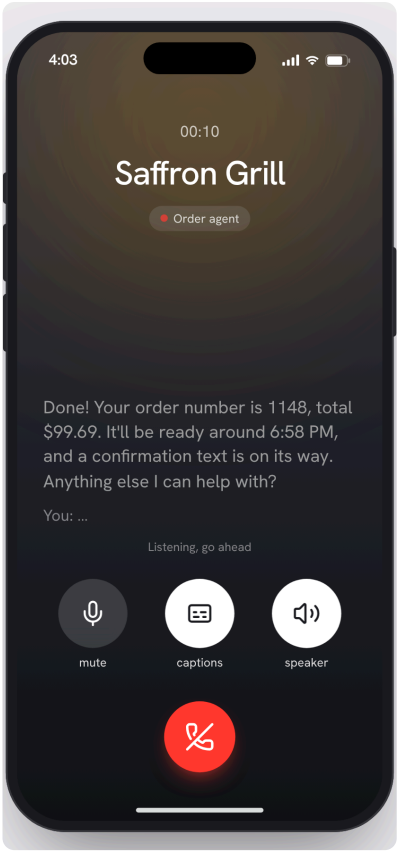
Figure 10. Crew console: talk to the agents by text or hands-free voice, and watch who is on the line, every handoff and every tool call, plus the texts sent to the caller.

Phone simulator

A dedicated page renders a realistic phone so the full caller experience can be demonstrated without a phone line: ring tone, call timer, the agent currently on the line, word-by-word live captions synced to the voice, and mute, captions and speaker controls. Orders and bookings made here go straight into the CRM.



Contact screen



Live call with captions

All CRM screens

SCREEN	PURPOSE
Overview	Tonight at a glance: KPIs, open orders, latest calls, tonight's bookings, 7-day revenue
Live orders	Kitchen board with chime, timers and one-tap status changes that text the customer
Reservations	Floor plan by time, bookings, seat / no-show / cancel, waitlist with automatic table offers
Menu	Prices, sold-out, upsell flags, allergens and dietary tags, all live for the agents
Call log	Transcripts, summaries, outcomes, agents, duration, recordings
Analytics	Trends, busy hours, top dishes, outcomes and agent usage
Talk to the crew	Test console with live trace and hands-free voice
Setup	Integration status, webhook URLs to copy, restaurant profile, text outbox, demo reset
Phone	Realistic phone simulator for demos and recordings

10 n8n automation workflows

> **In plain words:** n8n is the automation layer around the calls. It sends confirmations, reminders and the nightly report, and the restaurant can change those messages without a developer.

Seven importable workflows. Each starts with a Config node (app URL, shared secret, Twilio number) and checks the shared secret before doing anything.

#	WORKFLOW	TRIGGER	WHAT IT DOES
01	Menu, Caller & FAQ Lookup	Vapi tool call	Runs lookup_caller, get_menu, check_item, check_delivery_zone, search_knowledge_base and get_order_status; answers in Vapi's results format
02	Order Placement	Vapi tool call	price_order and place_order: the ticket hits the kitchen board
03	Reservation Manager	Vapi tool call	Availability, booking, changes, cancellations and the waitlist
04	Customer Notifications	Webhook from the app	Routes each message to Twilio SMS or WhatsApp: confirmations and status updates
05	Reservation Reminders	Every 15 minutes	Fetches bookings starting within two hours, texts each guest, marks them reminded
06	Post-Call Processing	Vapi end-of-call report	Stores transcript, recording, AI summary, outcome, agents and cost in the call log
07	Daily Owner Report	11 PM nightly	WhatsApps the owner the day's calls, orders, revenue, covers, upsell rate and top dishes

Two routing options for tools. By default Vapi calls the app's tool endpoint directly (one network hop, lowest latency). For clients who want every tool call visible and editable, the setup script can route each tool group through workflows 01–03 instead; the business logic is identical either way.

Workflow pattern (01–03)

1. Webhook receives Vapi's `tool-calls` message.
2. Config node supplies the app URL and secret; an IF node rejects requests without the right secret (401).
3. A Code node splits the tool-call list and checks each tool belongs to this workflow.
4. HTTP Request runs each tool on `/api/tools/execute`.
5. A Code node formats `{ results: [{ toolCallId, result }] }` and the workflow responds to Vapi.

11 Data model

> **In plain words:** every order, booking, customer and call lives in one database, so the history is complete and the CRM updates live as calls happen.

Nine Postgres tables in Supabase. Orders and reservations have Realtime enabled for the live board; row-level security is on, and the service key is only ever used server-side.

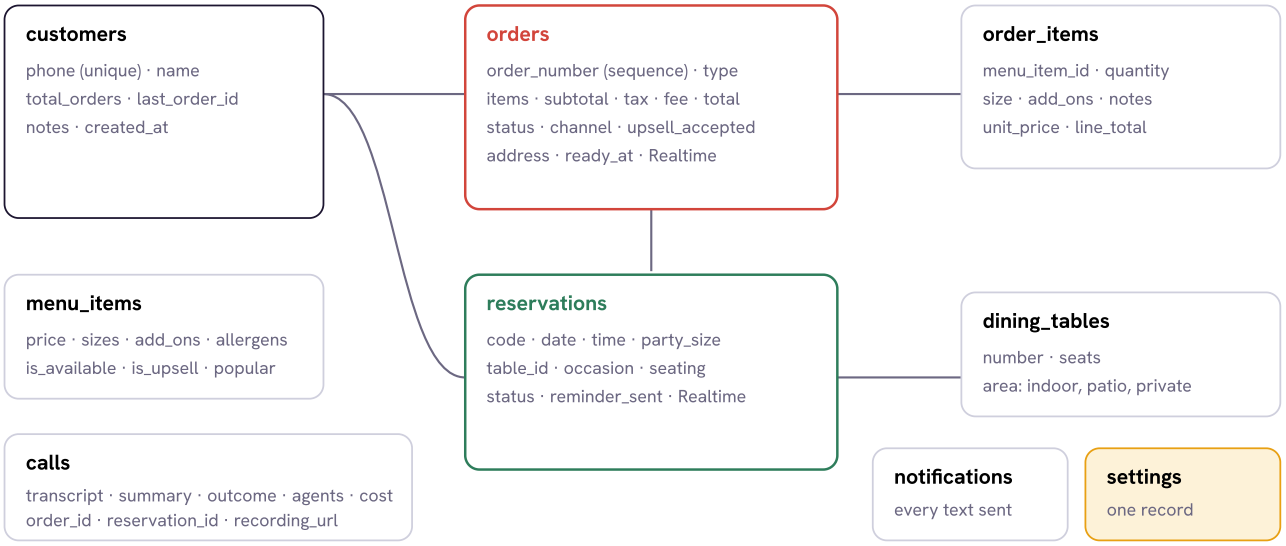


Figure 11. Data model. The single settings record (name, hours, tax, delivery zone, fees, transfer number) means the same build can be resold to any restaurant by editing one row.

- **Atomic order numbers** from a Postgres sequence via an RPC function, so two calls can never get the same number.
- **Line items in their own table** for reporting in SQL, plus a JSON copy on the order for fast reads by the board.
- **Triggers** keep `updated_at` current; a seeding script loads a week of realistic demo history for presentations.

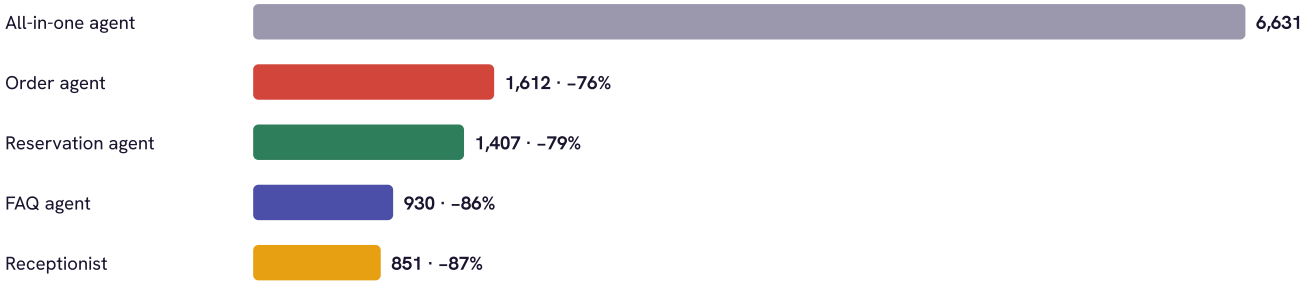
12 Cost efficiency

> **In plain words:** a typical 2–3 minute order call costs about 25 cents. The system keeps it that low by giving each agent only what it needs and keeping calls short and focused.

Voice AI is billed per minute and per token, so cost was treated as a design constraint, not an afterthought. Every measure below is implemented in the code.

Input tokens per AI call: specialists vs one all-in-one agent

Measured from the real prompts and tool schemas (characters ÷ 4). Lower is cheaper and faster.



All-in-one baseline = all four role prompts + all 15 tool schemas + the full menu embedded in the prompt.

MEASURE	HOW IT SAVES MONEY
Specialist agents, scoped tools	Each call carries only one role's prompt and its 3–7 tools instead of all 15: 76–86% fewer input tokens per call (chart above).
Menu via tools, not in the prompt	The 31-dish menu (≈2,300 tokens) is fetched only when needed and summarised by the tool, instead of being sent on every turn.
Instant routing without the model	Clear requests ("I want delivery", "book a table") are routed by code; a first message drops from 3 model calls to 2 (measured).
Rolling call memory, not full history	Tool results are condensed into short facts (max 14, about 300 characters each) carried between turns, instead of replaying full tool payloads.
Capped tool payloads	Tool responses sent back to the model are trimmed to 4,000 characters; lists are capped at 5–6 items, as a host would read them.
Summaries without extra AI calls	Phone calls reuse Vapi's built-in end-of-call summary; web conversations are summarised from tool results by code, with zero extra tokens.
Small, fast model with low reasoning	GPT-4o-mini on calls; "low" reasoning effort for reasoning models, since correctness comes from tools, not long thinking.
Billed minutes kept short	30-second silence timeout, 15-minute maximum, end-call phrases and short phone-style replies keep calls to about 2–3 minutes.
No wasted retries	Only overloads and timeouts are retried; wrong keys or bad requests fail fast. A circuit breaker pauses a slow provider for 60 seconds.
Zero-cost fallback and demos	The built-in rule-based crew handles demos, testing and provider outages with no model cost at all.
Free-tier friendly infrastructure	Supabase free tier, self-hosted n8n and Vercel's free plan cover a single restaurant comfortably.

Estimated cost per call minute

Built from Vapi's published per-minute rates for the configured stack. Figures vary slightly with the voice provider chosen.



≈ $0.09–0.10 per minute · a typical 2–3 minute order call ≈ $0.20–0.30

Rates from vapi.ai/pricing (Nov 2026): hosting $0.05/min, Deepgram $0.0095–0.0099, OpenAI from $0.0077, ElevenLabs-class voice $0.0146–0.0238, Twilio inbound $0.008.

What that means for a restaurant

At about $0.25 per order call, 40 phone orders a night cost around $10 in AI time. Against an average ticket of $40–45 and zero commission, compared with 15–30% on delivery apps, the crew pays for itself with the first two or three orders of the evening, while also answering every question call and booking for free.

The business figures are illustrative, based on the demo restaurant's average ticket; they are not measured client results.

13 Speed and reliability

> **In plain words:** the caller should never wait. If the AI provider is slow, a built-in backup answers instantly, so every reply arrives within seconds.

A caller will not wait. The system is built so that a reply always comes quickly, even when an AI provider is slow or down.

Hard reply budget

Every reply in the web runtime has a 9-second budget across all model calls. If the model hasn't finished, the built-in crew answers instead, so the caller is never left in silence.

Circuit breaker

After two slow or failed replies in a row, the provider is skipped for 60 seconds and answers come instantly from the built-in crew, then the AI is retried automatically.

Smart retries

Overloaded (429/503) and timed-out requests are retried with short backoff inside the budget; configuration errors fail fast instead of wasting time.

Built-in fallback crew

A deterministic engine runs the same four agents and the same tools, handling orders, bookings, menu questions, typos and speech-recognition spellings with replies in milliseconds.

Fast turn-taking

In the browser, end-of-speech is detected after about 0.9 seconds of silence, and unfinished speech is never dropped when the browser stops listening early.

Measured

Built-in crew replies in about 0.07 s server-side; with a deliberately slow AI provider, every reply still arrived within 9.0 s and later replies were instant.

On live phone calls Vapi streams speech-to-text, the model and text-to-speech together, so callers typically hear replies within one to two seconds; the safeguards above protect the web console and simulator, which call the model directly.

14 Safety, security and data

> **In plain words:** the agents only promise what the restaurant can deliver: real prices, real tables and real allergen information. Keys and customer data stay on the server.

Security

- Webhook endpoints for Vapi and n8n require a shared secret (`x-dineline-secret` or Vapi's server secret).
- Supabase row-level security is enabled with no public policies; the service-role key is used only on the server.
- No secrets in the repository: all keys live in `.env.local`, which is git-ignored; a documented `.env.example` is provided.
- Inputs are validated on the API endpoints (statuses, prices, session IDs, phone numbers).

Safe behaviour

- Allergen answers come only from menu data, with a clear cross-contact notice; agents never guess.
- Orders are placed only after a full read-back and a clear yes from the caller.
- Complaints, refunds, catering and very large parties go to a human by live transfer.
- Bookings are validated against opening hours, last seating, a 60-day window and table capacity.
- Delivery orders are checked against the delivery zone and minimum before they are placed.

15 Testing and verification

> **In plain words:** every flow was tested end to end before handover, from simple orders to awkward edge cases, and every total was checked by hand.

Every part of the system was exercised end to end before delivery. Where a live third-party account was not available in the build environment, the integration was verified against a faithful mock of that service's API.

AREA	HOW IT WAS VERIFIED	RESULT
Conversation flows	Scripted calls: delivery with upsell, allergen question then pickup, booking with alternatives, cancellation, hours and parking, transfer to staff	Pass
Edge cases	Booking details over several turns, full time slots, waitlist, large parties, booking changes, reorders, unknown dishes, item removal, typos, speech spellings, order changes before confirming	Pass
Arithmetic	Every read-back total checked by hand, e.g. $87.90 + $7.80 tax + $3.99 delivery = $99.69	Pass
n8n workflows	All 7 imported and run in a real n8n 2.35 instance against the app; Twilio sends stubbed pending credentials	Pass
Database	Schema, seed and a week of demo history loaded into a real Postgres engine, including re-runs	Pass
Vapi integration	Tool-call and end-of-call payloads in Vapi's format; setup script run end to end against a mock Vapi API (15 tools, 4 assistants, 1 squad)	Pass
AI runtime	Handoffs, tool calling, retries, 9-second cap and circuit breaker tested against mock OpenAI-compatible servers (fast, slow and failing)	Pass
Browser and voice	All CRM screens in headless Chrome with no console errors; hands-free calls and the phone simulator tested with simulated microphone and voice, including unfinished speech	Pass
Build quality	Strict TypeScript typecheck and production build	Pass
Go-live tooling	One-command Vapi setup (tools, assistants, squad, phone number), Supabase schema and seed scripts, Twilio and n8n connection settings, all documented in SETUP.md	Pass

16 Repository and setup

> **In plain words:** everything needed to run, change and extend the system is in one repository, with a step-by-step setup guide.

Repository layout

FOLDER	CONTENTS
`app/`	CRM screens and 19 API routes
`components/`	Kitchen tickets, crew chat, voice call, sidebar
`lib/agents/`	Agent prompts, LLM crew, built-in crew, sessions
`lib/tools.ts`	The 15 business tools
`lib/store/`	Supabase and local store behind one interface
`n8n/`	7 importable workflows
`supabase/`	schema.sql, seed.sql
`vapi/`	Squad setup script, exported squad config
`scripts/`	Crew tests, seeders, generators
`docs/`	Setup guide, diagrams, screenshots, this report

From zero to live

1. `npm install` and `npm run dev`: the CRM and crew run immediately in demo mode with a week of data.
2. Supabase: run `schema.sql` and `seed.sql`, add the keys, run `npm run seed:supabase`.
3. Deploy to Vercel (or tunnel locally) and set `PUBLIC_APP_URL`.
4. Vapi: add the key and run `npm run setup:vapi` to create the tools, four assistants and the squad, and attach the phone number.
5. n8n: import the 7 workflows, fill each Config node, connect Twilio, activate.
6. Make it yours: name, hours, tax, delivery zone and menu live in one settings record and the menu table.

17 Delivery and handover

> **In plain words:** this is the complete list of what the restaurant receives, how it was checked, and how the work was engineered so it is easy to run and extend.

What was handed over

- **Voice agents:** Receptionist, Order, Reservation and FAQ assistants in one Vapi Squad, created by a single setup command, with handoffs, staff transfer and end-call rules.
- **Business tools:** 15 tools for caller lookup, menu, pricing, delivery zones, orders, availability, bookings, waitlist, knowledge base and transfer, served to Vapi, n8n and the web console.
- **Automation:** 7 importable n8n workflows covering tool calls, notifications, booking reminders, post-call processing and the nightly owner report.
- **Custom CRM:** 9 screens covering the overview, kitchen board, reservations, menu, call log, analytics, crew console, setup and phone simulator.
- **Database:** Supabase schema with 9 tables, row-level security, Realtime, an order-number sequence, seed data and a week of demo history.
- **Prompts and scripts:** agent instructions, Vapi setup and export scripts, Supabase seeders and an automated crew test suite.
- **Documentation:** README, step-by-step setup guide, architecture diagram, screenshots and this delivery report.

How it was verified

- **Conversation flows** for orders, upsells, bookings, changes, cancellations, menu questions and staff transfers run end to end, with every total checked by hand.
- **All 7 n8n workflows** imported and run in a real n8n instance against the app.
- **Vapi setup script** run end to end, creating 15 tools, 4 assistants and the squad; tool-call and end-of-call payloads tested in Vapi's format.
- **Database** schema, seed and demo history loaded into a real Postgres engine; strict TypeScript typecheck and production build pass.

Engineering approach

- **One business layer:** phone, browser voice, web chat and n8n all call the same 15 tools, so behaviour is identical on every channel.
- **Configuration over code:** name, hours, tax, delivery zone and menu live in the database and change from the CRM with no redeploy.
- **Provider-agnostic AI:** any OpenAI-compatible model works, with a built-in crew as an instant fallback.
- **Built on free and low-cost tiers** of every service in the stack.

18 Key capabilities and operational impact

> **In plain words:** what the system means for the restaurant day to day: faster answers, exact orders, a full book and an owner who sees everything.

METRIC	WHAT THE SYSTEM DELIVERS
Speed to answer	Every call answered on the first ring, 24/7, including several calls at once during the dinner rush
Reply speed	Natural back-and-forth on calls; a hard cap of 9 seconds on any reply in the web console, with an instant fallback
Order accuracy	Items matched to the live menu, sizes and add-ons confirmed, and a full read-back with tax and fees before placing
Average ticket	One natural upsell per order, chosen from items that pair with what the caller ordered
Bookings	Best-fit table assigned automatically; nearest times offered when full; waitlist with automatic table offers
Kitchen handoff	Ticket on the kitchen board seconds after the caller says yes, with a chime and age timers
Customer follow-up	Order, ready, out-for-delivery, booking and reminder texts by SMS or WhatsApp, sent automatically
Owner visibility	Call log with transcripts and summaries, analytics, and a nightly WhatsApp report
Cost of a call	About $0.09–0.10 per minute, roughly $0.20–0.30 for a 2–3 minute order call; zero commission
Menu control	Prices, sold-out items and upsells change in the CRM and reach the agents on the very next call
Reliability	Retries, a circuit breaker and a built-in crew keep calls answered when an AI provider is slow
Quality	Every flow, edge case and total verified end to end before delivery

Cost figures are estimates from Vapi's published per-minute rates for the configured stack.

Muhammad Hassan

AI Automation Engineer & Full Stack Developer

Voice AI agents, multi-agent systems, n8n automation, custom CRMs and Shopify.